Universidad Nacional de Rosario

Facultad de Ciencias Exactas, Ingenieria Y Agrimensura

Introducción a la IA - R413

Trabajo Práctico I - Informe

Alumnos:

Juan Ignacio Steeman

Ernesto Jesús Savio

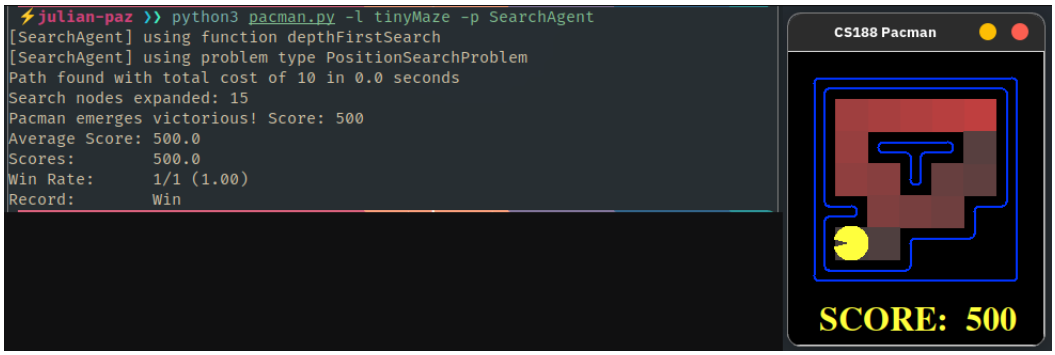
Julián Paz

Ejercicio 1

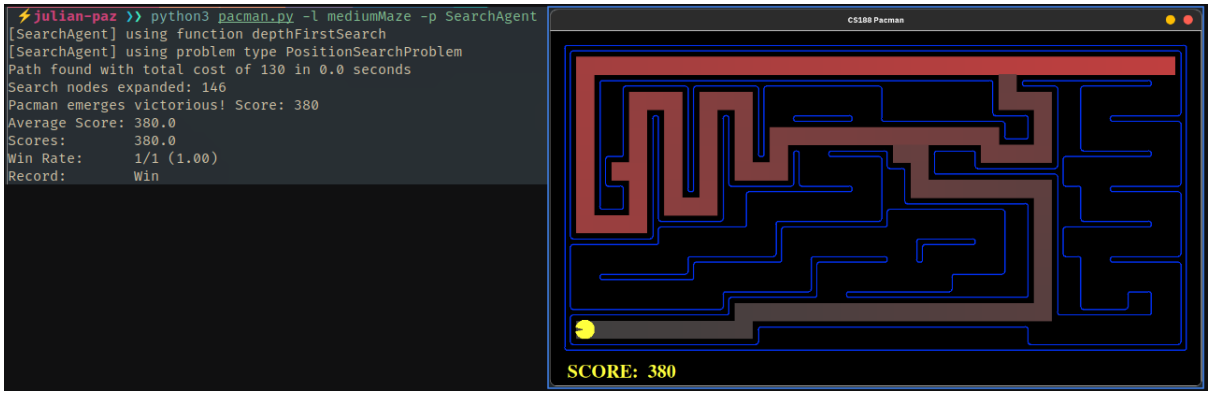
Implemente el algoritmo de búsqueda DFS (Depth-First Search) en la función llamada *depthFirstSearch* del archivo **search.py** (recuerde que la versión completa de DFS evita expandir estados ya visitados). Primero probamos ejecutar los siguientes casos:

- `python3 pacman.py -l tinyMaze -p SearchAgent`
- `python3 pacman.py -l mediumMaze -p SearchAgent`
- `python3 pacman.py -l bigMaze -z .5 -p SearchAgent`

En el primer caso obtuvimos el siguiente resultado:



Por otra parte, en el segundo caso obtuvimos el siguiente resultado:



Por último, en el tercer caso obtuvimos el siguiente resultado:

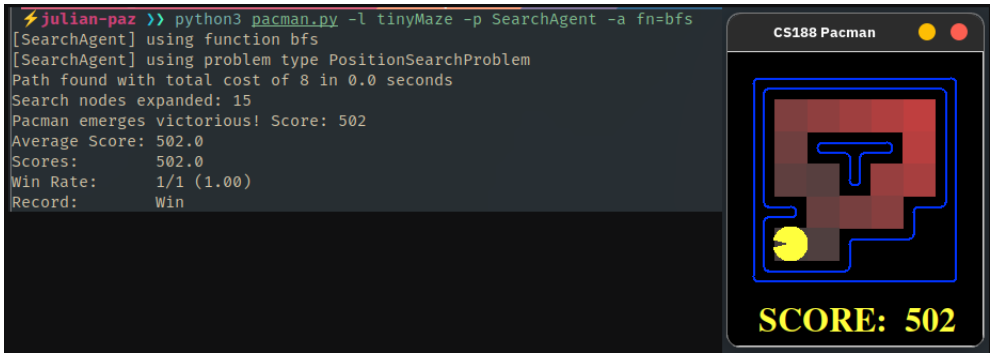
El orden de exploración que sigue el algoritmo de DFS es el siguiente: derecha, izquierda, abajo, arriba. Consideramos que es un orden exploración razonable, aunque puede llevar a un camino no óptimo (por ejemplo el primer caso). Más allá de ello, cualquier orden de exploración (en DFS) puede llevarnos a una solución no óptima.

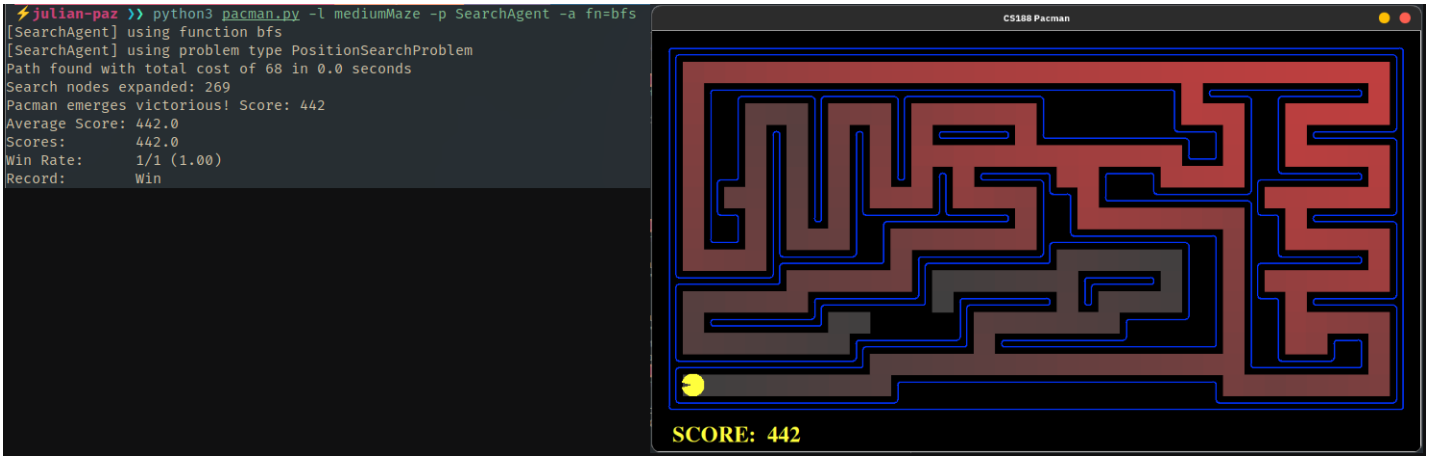
Pacman no necesariamente debe explorar todas las casillas, le basta con explorar hasta encontrar un camino a la meta. Además, Pacman no tiene que pasar por todas las casillas exploradas, ya que al encontrar un camino a la meta, solo deberá recorrer las casillas pertenecientes a dicho camino.

Ejercicio 2

Implemente el algoritmo BFS (Breadth-First Search) en la función llamada *breadthFirstSearch* en el archivo **search.py**. Testee su código de la misma forma que lo hizo para el DFS. ¿Encuentra DFS la solución de menor costo?

Volvimos a correr los mismos casos de prueba que usamos en DFS ahora en BFS, obteniendo así los siguientes resultados:





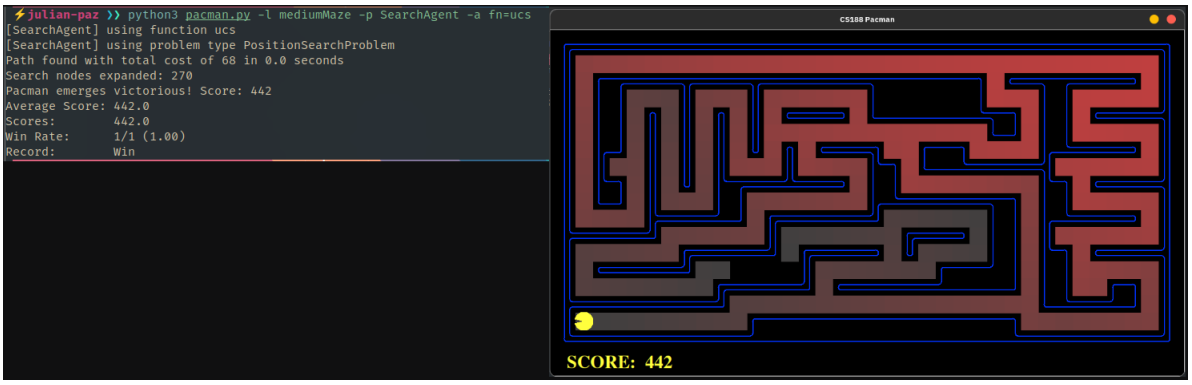
En todos los casos, BFS obtiene la solución de menor costo. Por otro lado, DFS no siempre encuentra la solución optima (los casos del ejercicio 1 son un ejemplo de ello).

Ejercicio 3

Implemente el algoritmo de búsqueda UCS (Uniform Cost Search) en la función *uniformCostSearch* del archivo **search.py** (recuerde utilizar lo que necesite de **util.py**). Una vez implementado observe la conducta del agente Pacman en los tres laberintos que se presentan a continuación, donde se modifica la función de costo que utiliza el agente (los agentes y las funciones de costo ya están implementadas):

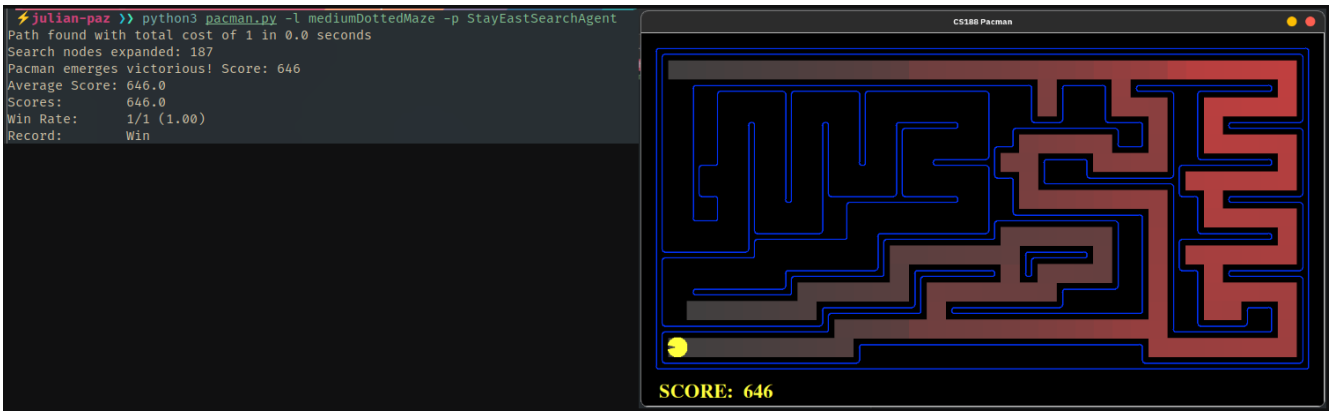
- `python3 pacman.py -l mediumMaze -p SearchAgent -a fn=ucs`
- `python3 pacman.py -l mediumDottedMaze -p StayEastSearchAgent`
- `python3 pacman.py -l mediumScaryMaze -p StayWestSearchAgent`

Observamos que en el primer laberinto se obtuvo el siguiente resultado:

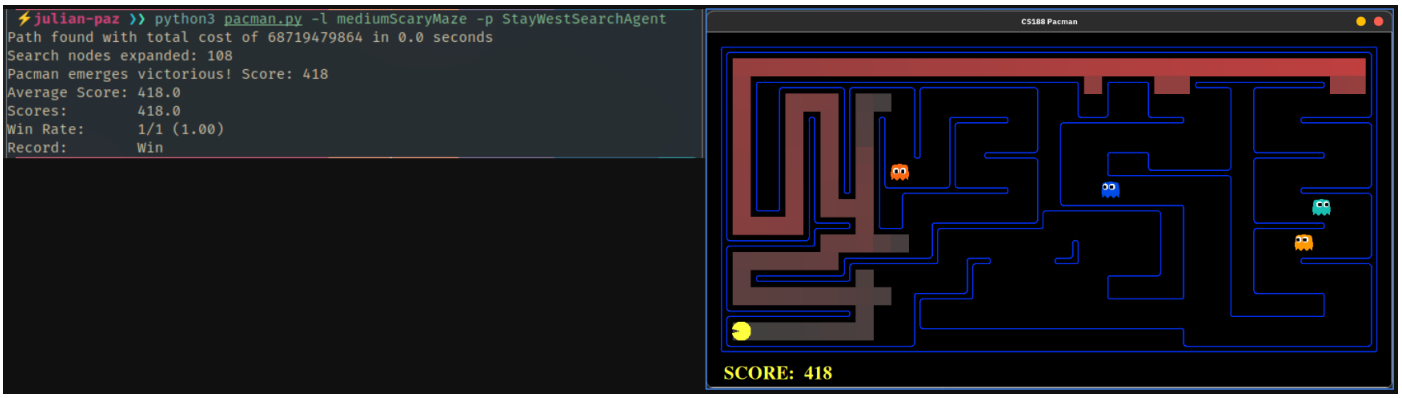


En este primer caso podemos observar que el Pacman obtiene la solución optima para el problema, y expande la misma cantidad de nodos que BFS.

Por otro lado, en el segundo caso:



Observamos que no sigue la misma ruta que antes, ya que toma la ruta que permite que el Pacman coma todas las pastillas del laberinto (para lograr esto se cambió la función de costo).
En en último laberinto podemos observar que:



En este último caso volvemos a observar que el Pacman toma una nueva ruta, esta nueva permite que el Pacman no se encuentre con los fantasmas (para lograr esto se cambió la función de costo).

Ejercicio 5

En el libro de Norvig y Russell "Artificial Intelligence: A Modern Approach, 4th edition" la implementación de A* se hace a partir del pseudocódigo de la Figura 3.7 "The best-first search algorithm". La misma desempata nodos repetidos según el costo de ruta o "PATH-COST". ¿Bajo qué condiciones de la función heurística hace falta hacer eso? Piense en qué pasa cuando la función heurística es consistente o admisible, y su aplicabilidad en grafos o árboles. Chequee de nuevo su implementación de A* para asegurarse que coincide con ésta.

Sabemos que el algoritmo A* es completo, es decir, siempre encontrará la ruta a la Meta. Ahora, el hecho de que este algoritmo sea óptimo depende de ciertas propiedades de nuestra heurística.

- Una heurística es **admisible** cuando nunca sobreestima el costo real de llegar a la meta más cercana, es decir: $h(n) \leq h^*(n)$, para todo nodo n, donde h es nuestra heurística y h^* es el costo real hasta la meta.

- Una heurística es **consistente** cuando para todo nodo n y su sucesor n' al que llegamos al realizar la acción a tenemos que: $h(n) \leq c(n, a, n') + h(n')$.

Si una heurística es consistente, entonces también será admisible, pero no vale la vuelta. La admisibilidad nos asegura que A* será óptimo buscando en árboles. Para grafos dependera de como está programado A*, si consideramos un conjunto de visitados y restringimos a visitar una unica vez cada estado tendremos que no sera optimo. Para recuperar la optimalidad hay que pedirle consistencia, propiedad que nos asegura que cada vez que alcancemos un estado el camino con el que llegamos hasta este será optimo.
Por otro lado, si consideramos un A* el cual permite actualizar el costo de un estado visitado (como el presentado en el libro e implementado en el TP) la admisibilidad nos asegurara que el algortimo es optimo y la consistencia simplemente nos brindara una mejora en la eficiencia del algoritmo. Esto es asi pues, cada vez que llegamos a un estado sabemos que ese camino es optimo y no hara falta en un futuro recorrerlo nuevamente, pues todos los estados fueron alcanzados con un costo optimo.
En la implementación de A* para desempatar nodos repetidos se mira el PATH-COST, esta hace falta hacerlo cuando la función herística es inconsistente por todo lo explicado arriba.
Para utilizar A* en un grafo nos conviene útilizar una heurística consistente.
FALTA VERIFICAR BIGMAZE VS UCS.
Análisis de las distintas estratégias de búsqueda para el laberinto "openMaze":

- **bfs:** Expande 682 nodos para llegar y llega con costo óptimo. Podemos ver, gracias al coloreado que muestra el laberinto, que BFS se comporta como lo esperado. Se exploran todos los nodos de manera uniforme con respecto a la profundidad en el grafo de busqueda.

- **dfs:** Expande 577 nodos pero no llega con el costo óptimo, coincidiendo con la teoria pues, DFS no es un algoritmo de costo optimo. Ademas, se puede observar a traves del laberinto como DFS explora por completo una rama incorrecta (pues se termina sin llegar a una meta) y luego como segunda rama elige una que lo lleva a Pacman a un estado meta.

- **ucs:** Expande 683 nodos y llega con costo óptimo (Dijkstra es optimo). Vemos que se comporta igual que BFS, esto es asi, pues si miramos la funcion $getSuccessors()$ todas las acciones tienen costo 1. Al realizar una busqueda acumulando el costo de llegar a cada nodo, donde cada nodo cuesta 1 obtenemos una busqueda uniforme en profundidad, que es exactamente BFS.
- **A*:** En A* como se comento anteriormente va a depender completamente de la heuristica. Si lo corremos sin heuristica estamos haciendo UCS, si le ponemos una heuristica consistente como manhattan vemos que llega de manera optima y no explora todo el espacio de estados (con una heuristica consistente nos aseguramos que al llegar por primera vez a un nodo, su camino es optimo).

Ejercicio 6

Implemente una heurística no trivial consistente para el CornersProblems en *cornersHeuristic*. Se puede construir una heurística consistente que expanda menos de 800 nodos. Desarrolle las heurísticas que probó, y cómo llegó o por qué la elegida es consistente.

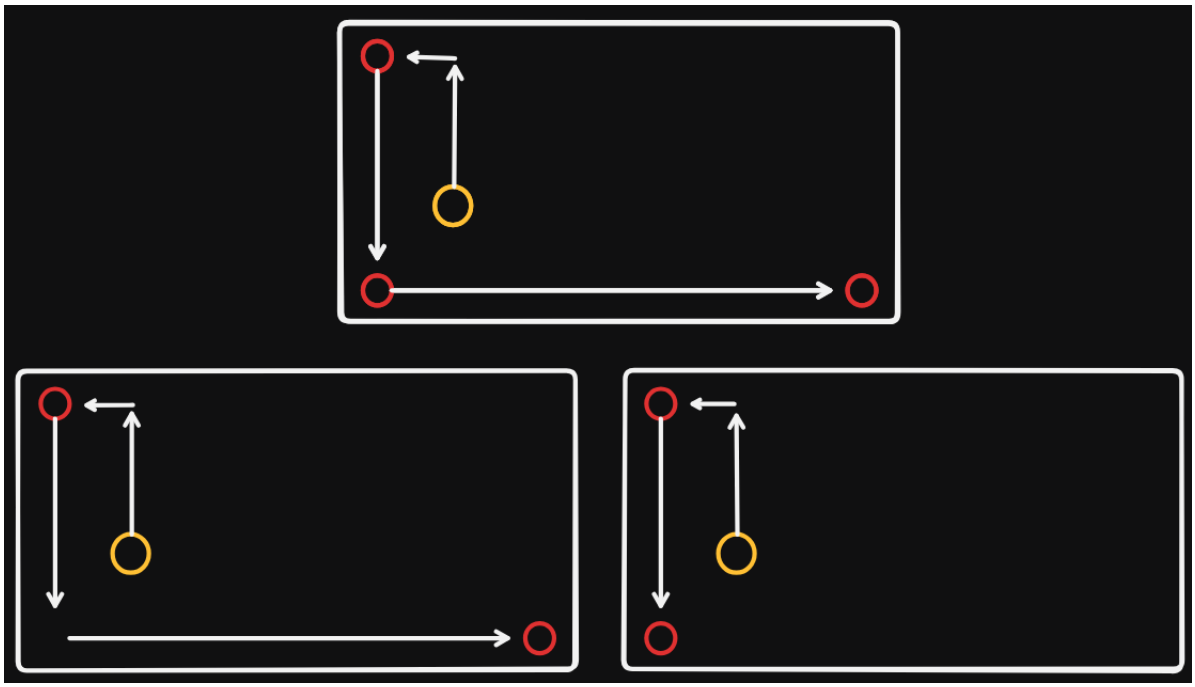
Para llegar a la heurística planteada lo primero que hicimos fue relajar el problema y tratar de buscar una heurística admisible en el problema relajado. Esto es, buscar una heurística admisible para el mismo problema pero en un laberinto sin paredes.
Para buscar una heurística admisible en este problema la distancia manhattan es una excelente herramienta pues nos da la distancia que debe recorrer el Pacman para ir de un punto a otro.
Como queremos acercarnos lo máximo posible al costo real sin sobreestimar, vamos a dividir el problema por casos:

- Todas las esquinas tienen comida.
- Tres de las cuatro esquinas tienen comida.
- Dos de las cuatro las esquinas tienen comida.
- Solo una esquina tiene comida.

En el primer caso, la solución óptima del problema es al menos igual a la suma de la distancia máxima entre dos pastillas, la distancia mínima de una pastilla al Pacman, y la distancia mínima entre dos pastillas:

Recordemos que todo el tiempo estamos hablando de distancias de manhattan.

En el segundo y tercer caso tenemos que la solución óptima es al menos igual a la suma de la distancia máxima entre dos pastillas y la distancia mínima entre un nodo y el Pacman:



El en último caso tenemos que la solución óptima es al menos igual a la distancia entre el Pacman y la pastilla:

Habiendo comentado esto, veamos la definición formal de la heurística:

$$h(n) = \begin{cases} maxDistance(n) + minDistance(n) + min_p(n) & si\ k = 4 \\ maxDistance(n) + min_p(n) & si\ k = 2, 3 \\ min_p(n) & si\ k = 1 \end{cases}$$

Donde $maxDistance(n)$ es la máxima distancia entre dos pastillas dentro del nodo n, $minDistance(n)$ es la mínima distancia entre dos pastillas dentro del nodo n, y $min_p(n)$ la mínima distancia entre una pastilla y el Pacman.

Determinemos si la misma es o no consistente/admisible. Para ello basta con probar que la heurístca es consistente, ya que consistencia implica admisibilidad.
Para probar consistencia debemos ver que para todo nodo n y para todo sucesor n' de n alcanzable por la acción a, se verifica que:

$$h(n) \leq c(n, a, n') + h(n')$$

En este caso en particular, como todas nuestras acciones tienen coste 1, resulta que debemos probar que:

$$h(n) \leq h(n') + 1$$

Vamos a dividir esta prueba por casos:

- El Pacman come una pastilla al desplazarse.
- El Pacman no come una pastilla al desplazarse.

Primer caso: El Pacman come una pastilla al desplazarse.
Supongamos que el Pacman come una pastilla al desplazarse. Supongamos que en n el tablero tiene k pastillas, con $k \in [1.,4]$. Analicemos cada caso de k en particular.
Supongamos que $k = 4$:

- Veamos que:

$$h(n) \leq h(n') + 1$$

- Observemos:
  - $h(n) = maxDistance(n) + minDistance(n) + min_p(n)$
  - $h(n') = maxDistance(n') + min_p(n')$
- Por un lado tenemos que $min_p(n) = 1$ ya que el Pacman come al desplazarse (i).
- Por otra parte tenemos que $min_p(n') \geq minDistance(n)$ ya que el Pacman en n' se encuentra en la posición de la pastilla que comió (ii).
- Por último, observemos que $maxDistance(n) = maxDistance(n')$ ya que en n' quedan dos pastillas en esquinas opuestas. (iii)
- Debido a ello:

$$h(n) =_{(def\ \ h(n))} maxDistance(n) + minDistance(n) + min_p(n) =_{(iii),\ (i)}$$
$$maxDistance(n') + minDistance(n) + 1 \leq_{(ii)} maxDistance(n') + min_p(n') + 1 = h(n') + 1$$

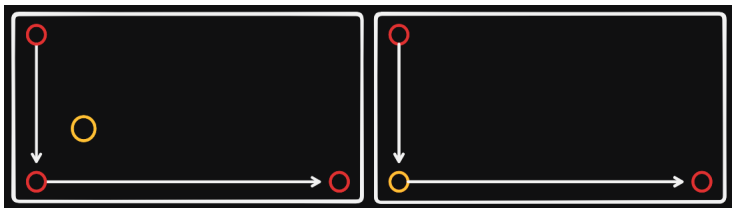
- Pasando en limpio, tenemos que:

$$h(n) \leq h(n') + 1$$

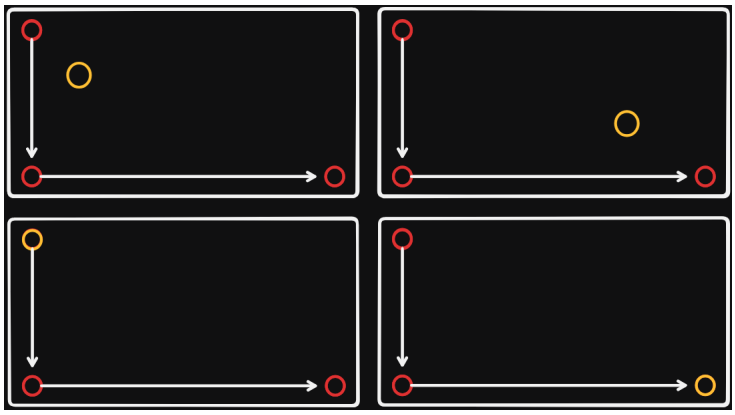
- Por lo tanto si $k = 4$, vale consistencia.

Supongamos que $k = 3$:

- Si $k = 3$, tenemos que:
  - $h(n) = maxDistance(n) + min_p(n)$
  - $h(n') = maxDistance(n') + min_p(n')$
- Por un lado, tenemos que $min_p(n) = 1$, ya que el Pacman puede comer al desplazarse. (i)
- Por otra parte observemos que en n' el Pacman se encuentra en la posición donde estaba la pastilla que comió.
- Notemos que si Pacman comió la pastilla adyacente entre las otras dos pastillas, entonces $maxDistance(n) = maxDistance(n')$, por ende tenemos que se verifica la desigualdad $maxDistance(n) \leq maxDistance(n') + min_p(n')$.



- Notemos que si Pacman comió una de las pastillas de las esquinas que solo tienen una pastilla adyacente, entonces $maxDistance(n) \geq maxDistance(n')$. Pero observemos que en este caso se verifica que $maxDistance(n) = min_p(n') + maxDistance(n')$.



- Por lo tanto podemos afirmar que $maxDistance(n) \leq maxDistance(n') + min_p(n')$. (ii)
- Luego:

$$h(n) =_{(def\ h(n))} maxDistance(n) + min_p(n) =_{(i)} maxDistance(n) + 1$$
$$\leq_{(ii)} maxDistance(n') + min_p(n') + 1 =_{(def\ h(n'))} h(n') + 1$$

- Pasando en limpio tenemos que:

$$h(n) \leq_{(ii)} h(n') + 1$$

- Por lo que vale consistencia si $k = 3$

Supongamos que $k = 2$:

- Si $k = 2$, tenemos que:
  - $h(n) = maxDistance(n) + min_p(n)$
  - $h(n') = min_p(n')$
- En este caso tenemos que $min_p(n) = 1$ ya que el Pacman puede comer una pastilla al desplazarse. (i)
- Por otro lado, observemos que $min_p(n') = maxDistance(n)$ ya que el Pacman (en n') se encuentra en la posición de la pastilla (que en n) se encontraba a $maxDistance(n)$ de la pastilla restante. Por lo tanto tenemos que vale $maxDistance(n) = min_p(n')$ (ii)
- Por lo tanto, tenemos que:

$$\begin{aligned} h(n) =_{(def\ \ h(n))} maxDistance(n) + min_p(n) =_{(i)} maxDistance(n) + 1 \\ \leq_{(ii)} min_p(n') + 1 =_{(def\ \ h(n'))} h(n') + 1 \end{aligned}$$

- Pasando en limpio tenemos que:

$$h(n) \leq h(n') + 1$$

- Por lo tanto vale consistencia para $k = 2$

Para cerrar con el primer caso de la prueba, veamos que vale consistencia cuando $k = 1$:

- Si $k = 1$, entonces:
  - $h(n) = min_p(n)$
  - $h(n') = 0$ (esto se deduce de que solo quedaba una pastilla en n y con la acción a la comimos)
- Observemos que $min_p(n) = 1$ ya que el Pacman puede comer la pastilla al desplazarse. (i)
- Luego tenemos que:

$$h(n) =_{(def\ \ h(n))} min_p(n) =_{(i)} 1 = 0 + 1 =_{(def\ \ h(n'))} h(n') + 1$$

- Por lo tanto tenemos que vale que:

$$h(n) \leq h(n') + 1$$

- Por lo tanto vale consistencia si $k = 1$

Por lo tanto si el Pacman come una pastilla al realizar la acción a, resulta que se verifica que:

$$h(n) \leq h(n') + 1$$

Segundo caso: El Pacman no come una pastilla al desplazarse.

Si el Pacman no come una pastilla al desplazarse, entonces podemos afirmar que la cantidad de pastillas en n es igual a la cantidad de pastillas en n'. Supongamos que el tablero en n tiene k pastillas. Analicemos cada caso de k en particular.
Supongamos que $k = 4$

- Observemos que como no cambian la cantidad de pastillas de n a n', resulta que:
  - $h(n) = maxDistance(n) + minDistance(n) + min_p(n)$
  - $h(n') = maxDistance(n') + minDistance(n') + min_p(n')$
- Veamos que:

$$h(n) \leq h(n') + 1$$

- Observemos que como no se comen pastillas, se verifica que:
  - $maxDistance(n) = maxDistance(n')$
  - $minDistance(n) = minDistance(n')$
- Por otro lado, tenemos que $min_(p) = min_p(n') \pm 1$ ya que el Pacman se desplaza solo se mueve una casilla.
- Uno podría pensar que al desplazarse, la pastilla x para la cual $min_p(n) = dist(p, x)$ cambie (es decir, en n existe otra pastilla x' tal que $min_p(n') = dist(p, x')$) y que por consecuencia la diferencia entre $min_p(n)$ y $min_p(n')$ sea mayor a 1. Supongamos sin perder generalidad que $min_p(n') = min_p(n) - t$, con $t \geq 2$.
- Ahora bien, si eso fuera cierto, entonces la distancia entre p y x' en n debería ser igual a $dist(p, x') - 1$. En consecuencia existe x' en n talque $dist(p, x') < min_p(n)$ lo cual es absurdo. Por lo tanto $min_p(n) = min_p(n') \pm 1$.
- Luego, si $min_p(n) = min_p(n') + 1$ tenemos que:

$$h(n) = maxDistance(n) + minDistance(n) + min_p(n) =$$
$$maxDistance(n') + minDistance(n') + min_p(n') + 1 = h(n') + 1$$

- Si por otro lado $min_p(n) = min_p(n') - 1$ tenemos que:

$$h(n) = maxDistance(n) + minDistance(n) + min_p(n) =$$
$$maxDistance(n') + minDistance(n') + min_p(n') - 1 \leq h(n') + 1$$

- Por lo tanto resulta que:

$$h(n) \leq h(n') + 1$$

- En consecuencia vale consistencia si $k = 4$ (los demás casos son análogos).

Por lo tanto si el Pacman no come una pastilla al desplazarse, resulta que vale:

$$h(n) = h(n') + 1$$

Luego como se mantiene consistencia en ambos casos, resulta que se verifica la consistencia para la heurística h.

Ejercicio 7

Implementar en *foodHeuristic*, en el archivo **searchAgents.py**, una heurística para el FoodSearchProblem capaz de comer toda la comida en el menor tiempo posible. Explique y desarrolle el porqué de elegir esta heurística.

Para este ejercicio desarrollamos la heurística "remainingFoodHeuristic".
Esta heurística utiliza la cantidad de pastillas restantes del tablero:

$$h(n) = lenght(pastillas)$$

Donde *pastillas* es el conjunto conformado por todas las pastillas del tablero.
Esta solución es consistente (y por lo tanto admisible). Probemoslo.
Debemos probar que para todo nodo n y para todo nodo n' sucesor de n alcanzado por la acción a se verifica que:

$$h(n) \leq c(n, a, n') + h(n')$$

Luego, como toda acción (dentro de nuestro problema) tiene coste 1, resulta que lo que debemos probar es:

$$h(n) \leq h(n') + 1$$

Notemos que como nuestro conjunto de acciones consiste en que el Pacman se desplace a una posición adyacente, tenemos que la posición del Pacman en n' es adyacente a la posición del Pacman en n. A su vez esto nos permite afirmar que la cantidad de pastillas en n solo puede disminuir en una unidad en n' (ya que el Pacman solo se movió una casilla y a lo sumo hay una pastilla en una casilla).
Para probar consistencia vamos a hacer un análisis por casos:

- Al desplazarse el Pacman comió una pastilla.
- Al desplazarse el Pacman no comió una pastilla.

Primer caso: Al desplazarse el Pacman comió una pastilla.
Supongamos sin perder generalidad que la cantidad de pastillas en n es igual a $i > 0$.
Por definición de h, tenemos que la heurística en n es igual a i. En símbolos:

$$h(n) = i$$

Ahora bien, en n' tenemos que la cantidad de pastillas es $i - 1$ ya que estamos en el caso donde el Pacman comió una pastilla al desplazarse. En consecuencia tenemos que la heurística en n' es igual a $i - 1$. En símbolos:

$$h(n') = i - 1$$

Luego tenemos que:

$$h(n) = h(n) + 1 - 1 = (h(n) - 1) + 1 = h(n') + 1$$

Por lo tanto tenemos que $h(n) \leq h(n') + 1$, que es lo que quería probar.
Segundo caso: Al desplazarse el Pacman no comió una pastilla.
Supongamos sin perder generalidad que la cantidad de pastillas en n es igual a $i > 0$.
Nuevamente, por definición de h tenemos que:

$$h(n) = i$$

Ahora bien, como en n' el Pacman no comió ninguna pastilla, tenemos que la cantidad de pastillas en n' es i. Por consecuencia la heurística en n' es igual a:

$$h(n') = i$$

Luego tenemos que:

$$h(n) \leq h(n) + 1 = h(n') + 1$$

Por lo tanto tenemos que $h(n) \leq h(n') + 1$, que es lo que quería probar.
Luego como sin importar el caso llegamos a que:

$$h(n) \leq h(n') + 1$$

Resulta que h es una heurística consistente.

Otra idea

Veamos una mejor heurística. La idea va a ser realizar Prim al grafo conformado por las pastillas del tablero, y tomar como heurística el peso de dicho árbol recubridor mínimo (la suma de las aristas). El peso de las aristas estará dado por la distancia de manhattan.
Calcular Prim tiene un costo aproximado de n^2. Pero gracias a que en este ejercicios podemos llevar un diccionario con información de la heurística, podemos aprovecharnos de esto para que el costo de calcular la heurística en cada estado del tablero sea mucho menor.
Vamos a calcular Prim por primera vez y guardar la distancia entre pastillas en el diccionario, luego en cada estado se calculará nuevamente el algoritmo de Prim, pero esta vez no hara falta calcular las distancias, simplemente acceder al diccionario y buscar la mínima.
Sin embargo, todavia necesitamos acceder a la distancia entre las pastillas restantes, y dentro del algoritmo de Prim buscar la distancia mínima de cada pastilla (entre las pastillas accedidas). Pareciendo que el costo de calcular la heurística no disminuye demasiado. Por esto último (agregado a la complejidad que supone implementar esta heurística) es que decidimos no implementarla.